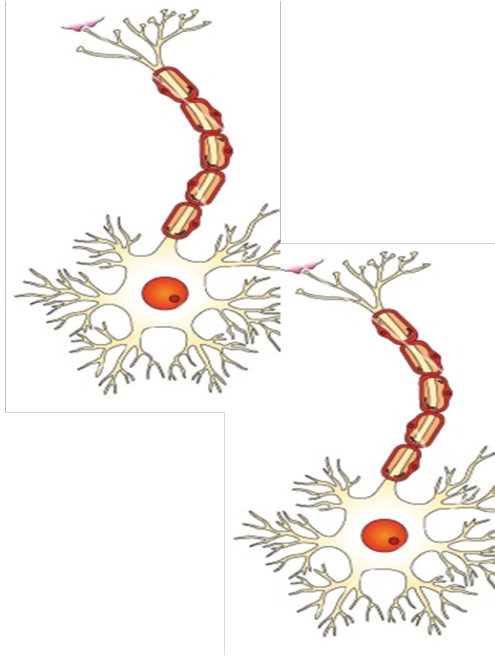


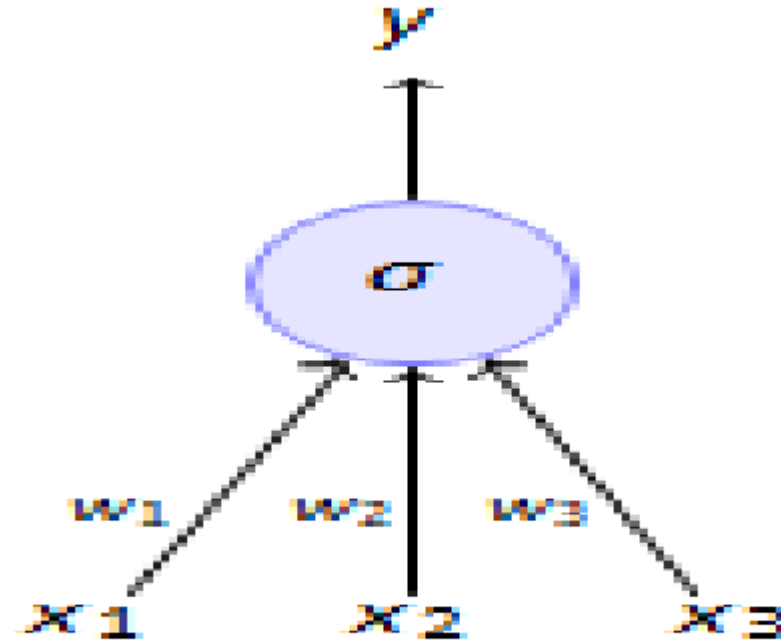


McCulloch Pitts Neuron & Perceptron

Artificial Neuron

The most fundamental unit of a deep neural network is called an *artificial neuron*

The inspiration comes from biological neurons



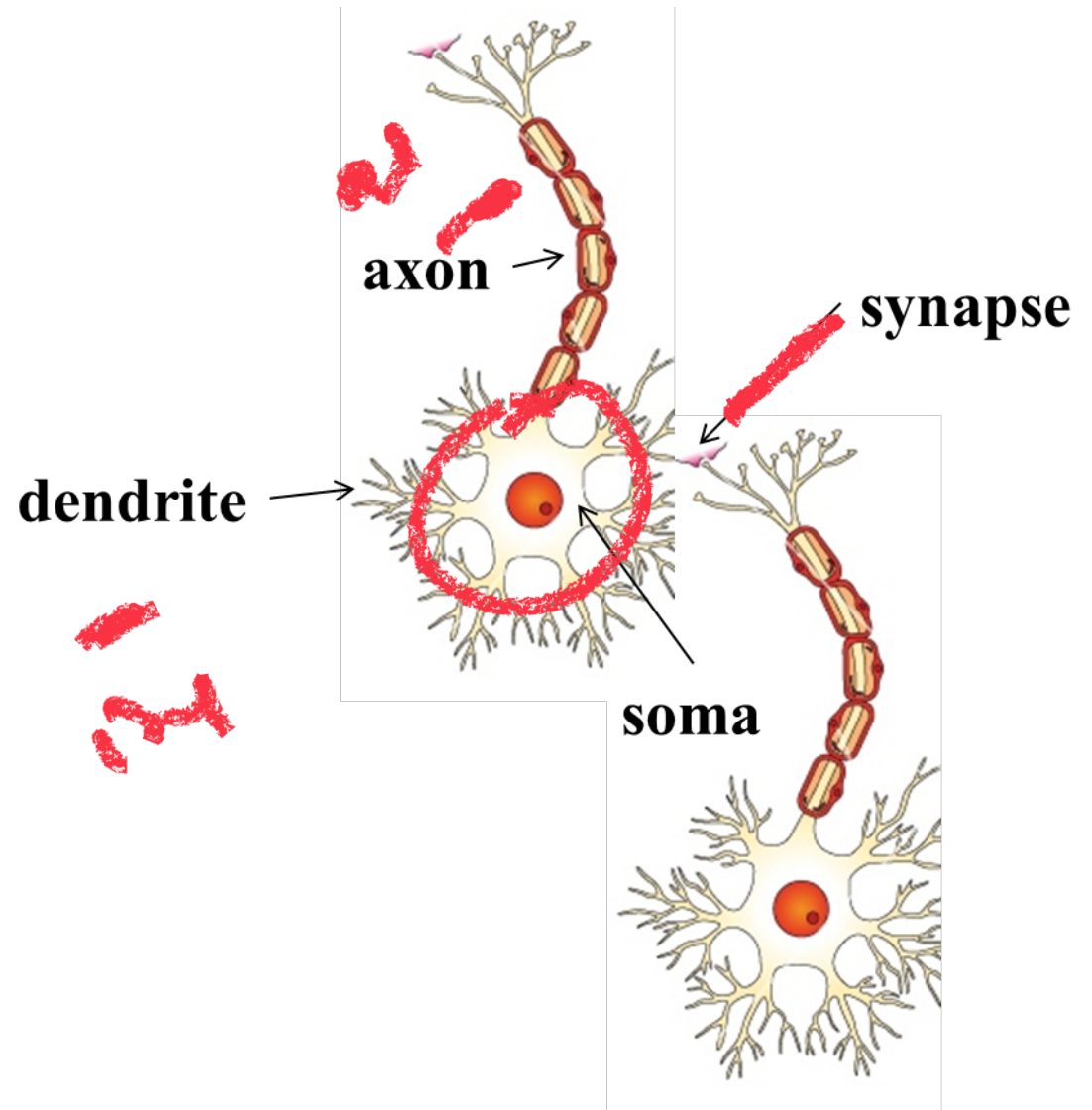
Biological Neuron

dendrite: receives signals from other neurons

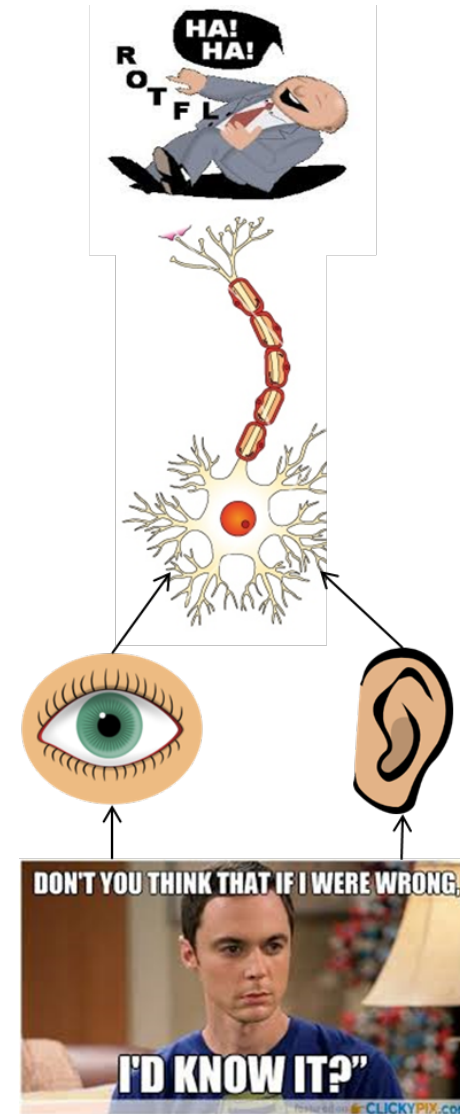
synapse: point of connection to other neurons

soma: processes the information

axon: transmits the output of this neuron



- Let us see a very cartoonish illustration of how a neuron works
- Our sense organs interact with the outside world
- They relay information to the neurons
- The neurons (may) get activated and produces a response (laughter in this case)



Of course, in reality, it is not just a single neuron which does all this

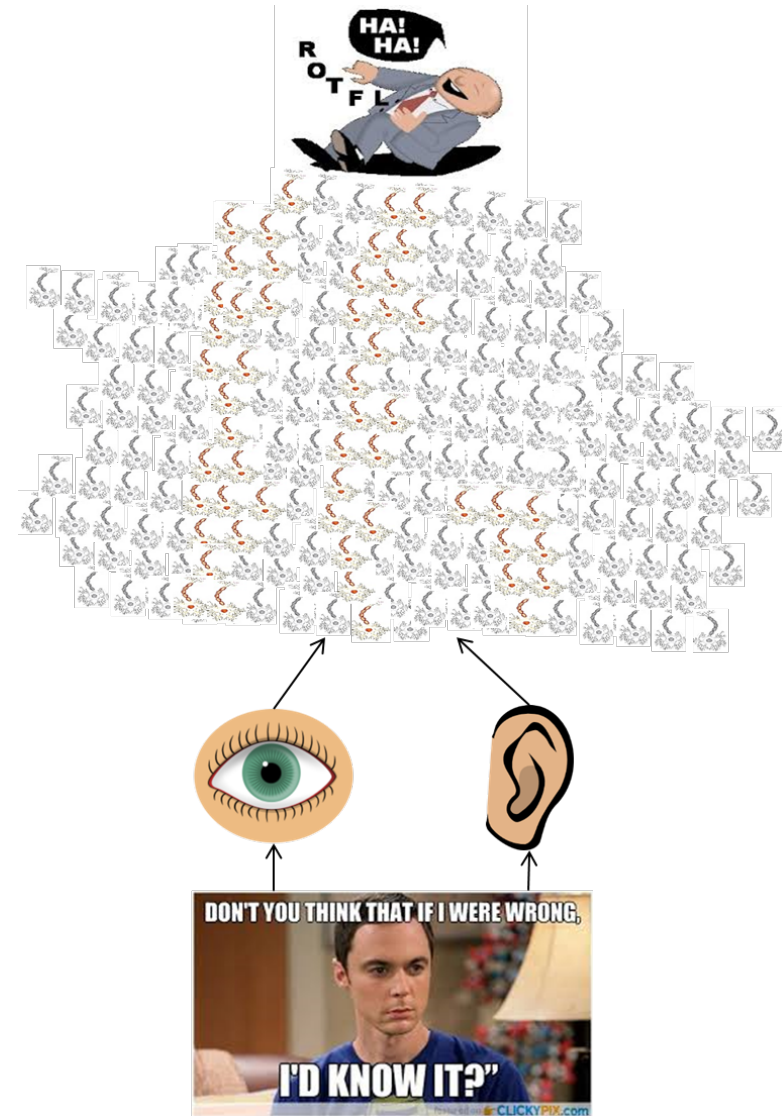
There is a massively parallel interconnected network of neurons

The sense organs relay information to the lowest layer of neurons

Some of these neurons may fire (in red) in response to this information and in turn relay information to other neurons they are connected to

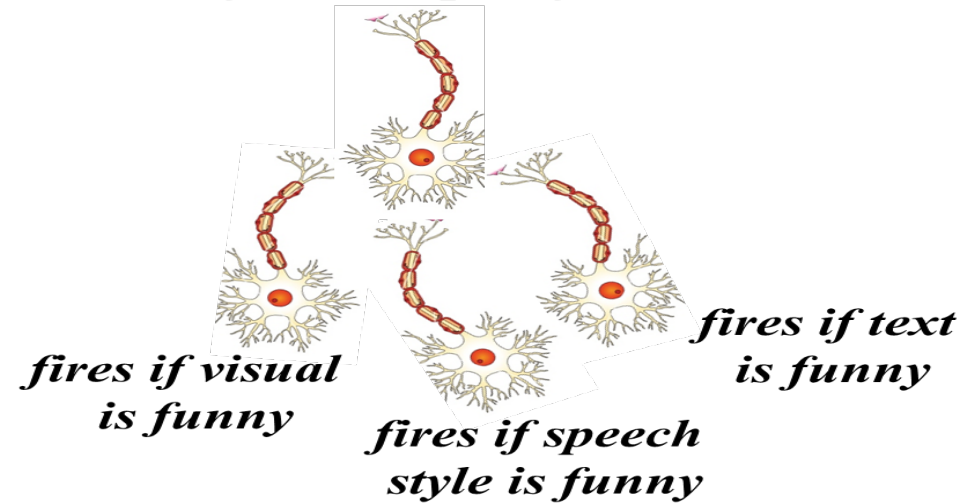
These neurons may also fire (again, in red) and the process continues eventually resulting in a response (laughter in this case)

An average human brain has around 10^{11} (100 billion) neurons!



This massively parallel network also ensures that there is division of work
Each neuron may perform a certain role or respond to a certain stimulus

*fires if at least
2 of the 3 inputs fired*



McCulloch Pitts Neuron

McCulloch (neuroscientist) and Pitts (logician) proposed a highly simplified computational model of the neuron (1943)

g aggregates the inputs and the function f takes a decision based on this aggregation

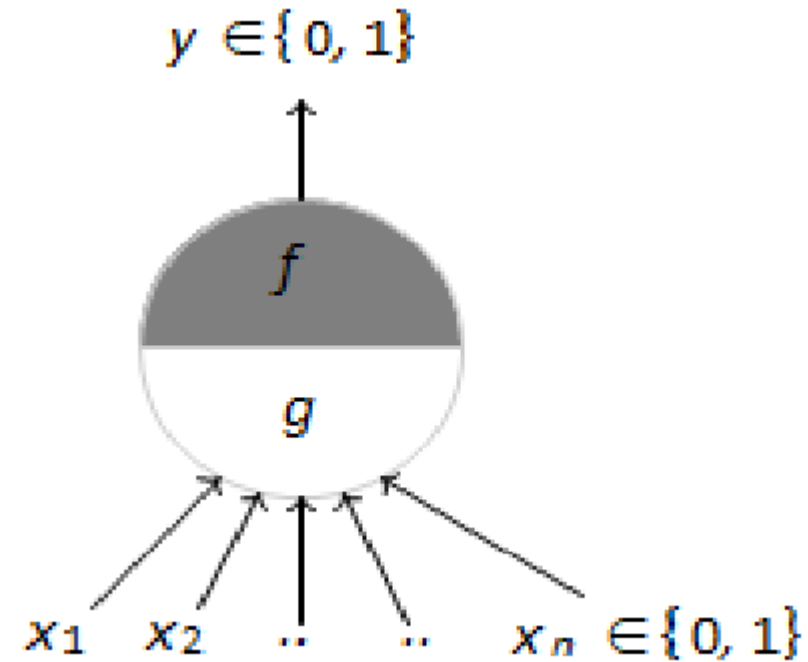
The inputs can be excitatory or inhibitory

$y = 0$ if any x_i is inhibitory, else

$$g(x_1, x_2, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n x_i$$

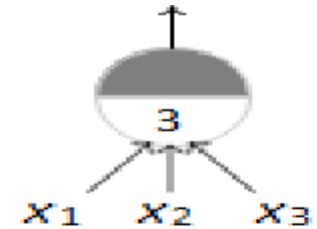
$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{if } g(\mathbf{x}) \geq \vartheta \\ 0 & \text{if } g(\mathbf{x}) < \vartheta \end{cases}$$

ϑ is called the
thresholding parameter
This is called Thresholding Logic



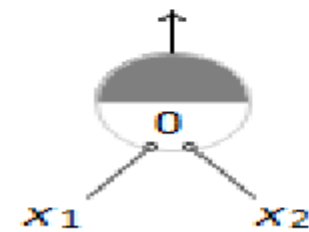
Implement some boolean functions using this McCulloch Pitts (MP) neuron

circle at the end indicates inhibitory input: if any inhibitory input is 1 the output will be 0

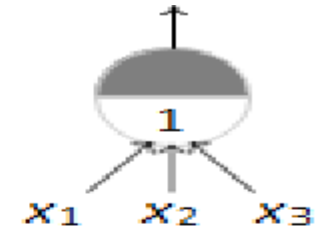


AND function

$$y \in \{0, 1\}$$



NOR function



OR function

$$y \in \{0, 1\}$$



NOT function

Geometric interpretation of a MP unit

A single MP neuron splits the input points (4 points for 2 binary inputs) into two halves

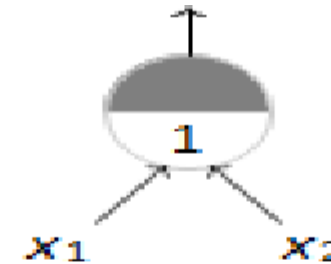
Points lying on or above the line

$\sum_{i=1}^n x_i - \theta = 0$ and points lying below this line

In other words, all inputs which produce an output 0 will be on one side $(\sum_{i=1}^n x_i < \theta)$

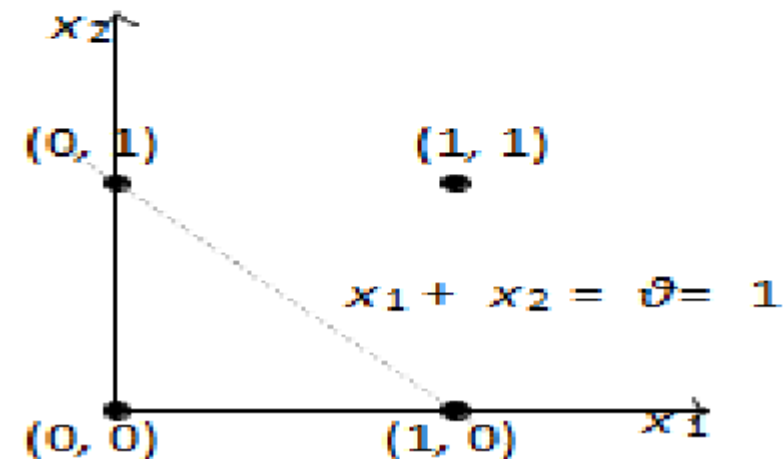
of the line and all inputs which produce an output 1 will lie on the other side

$(\sum_{i=1}^n x_i \geq \theta)$ of this line



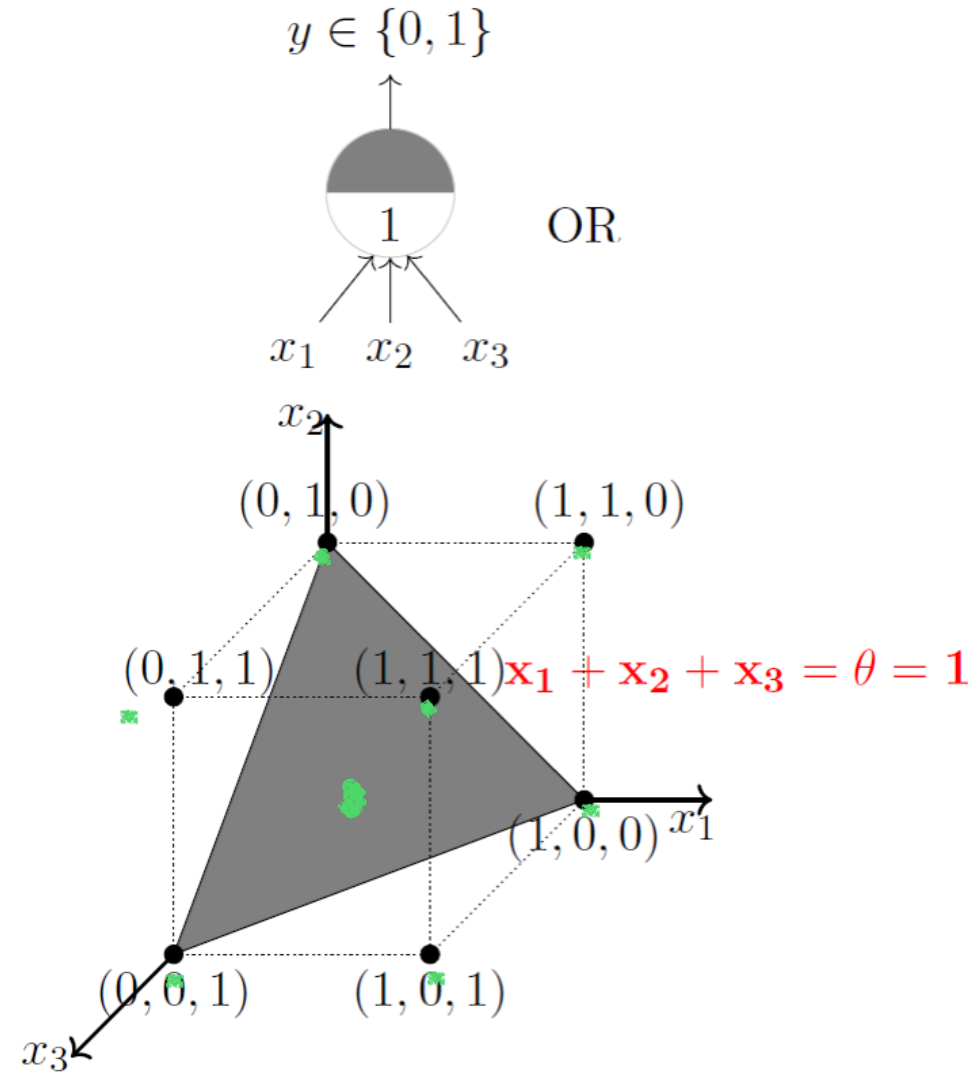
OR function

$$x_1 + x_2 = \sum_{i=1}^2 x_i \geq 1$$



More than two inputs

- Instead of a line we will have a plane
- For the OR function, we want a plane such that the point $(0,0,0)$ lies on one side and the remaining 7 points lie on the other side of the plane



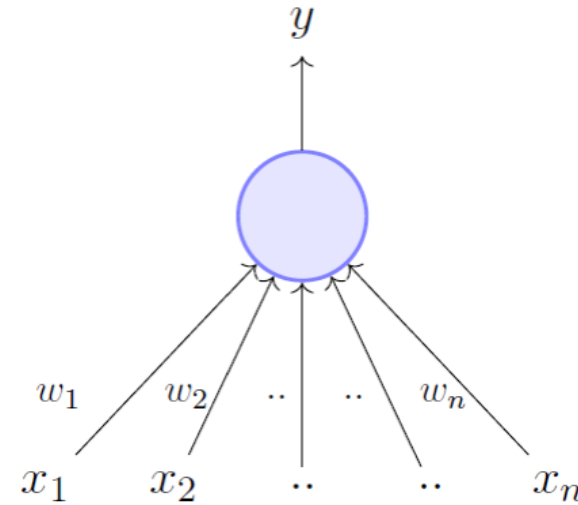
McCulloch Pitts Neuron

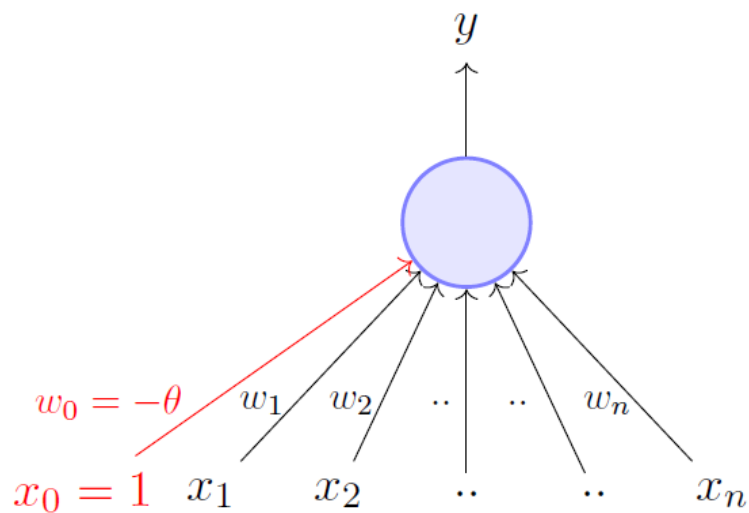
A single McCulloch Pitts Neuron can be used to represent boolean functions which are linearly separable

Linear separability (for boolean functions) : There exists a line (plane) such that all inputs which produce a 1 lie on one side of the line (plane) and all inputs which produce a 0 lie on other side of the line (plane)

Perceptron

- Frank Rosenblatt, an American psychologist, proposed the **classical perceptron** model (1958)
- A more general computational model than McCulloch–Pitts neurons
- **Main differences:**
 - Introduction of numerical weights for inputs and a mechanism for learning these weights
 - Inputs are no longer limited to boolean values
- Refined and carefully analyzed by Minsky and Papert (1969) - their model is referred to as the **perceptron** model here





A more accepted convention,

$$\begin{aligned}
 y &= 1 && \text{if } \sum_{i=0}^n w_i * x_i \geq 0 \\
 &= 0 && \text{if } \sum_{i=0}^n w_i * x_i < 0
 \end{aligned}$$

where, $x_0 = 1$ and $w_0 = -\theta$

$$\begin{aligned}
 y &= 1 && \text{if } \sum_{i=1}^n w_i * x_i \geq \theta \\
 &= 0 && \text{if } \sum_{i=1}^n w_i * x_i < \theta
 \end{aligned}$$

Rewriting the above,

$$\begin{aligned}
 y &= 1 && \text{if } \sum_{i=1}^n w_i * x_i - \theta \geq 0 \\
 &= 0 && \text{if } \sum_{i=1}^n w_i * x_i - \theta < 0
 \end{aligned}$$

McCulloch Pitts Neuron Vs Perceptron

- From the equations it should be clear that even a perceptron separates the input space into two halves
- All inputs which produce a 1 lie on one side and all inputs which produce a 0 lie on the other side
- In other words, a single perceptron can only be used to implement linearly separable functions
- Then what is the difference? The weights (including threshold) can be learned and the inputs can be real valued

McCulloch Pitts Neuron
(assuming no inhibitory inputs)

$$y = 1 \quad \text{if} \quad \sum_{i=0}^n x_i \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=0}^n x_i < 0$$

Perceptron

$$y = 1 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i < 0$$

Implementation of OR function

x_1	x_2	OR	
0	0	0	$w_0 + \sum_{i=1}^2 w_i x_i < 0$
1	0	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
0	1	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
1	1	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$

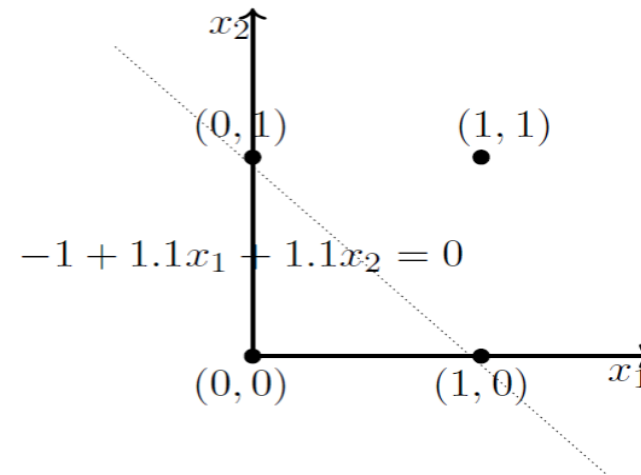
$$w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \implies w_0 < 0$$

$$w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \implies w_2 > -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \implies w_1 > -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 \implies w_1 + w_2 > -w_0$$

- One possible solution to this set of inequalities is $w_0 = -1, w_1 = 1.1, w_2 = 1.1$ (and various other solutions are possible)

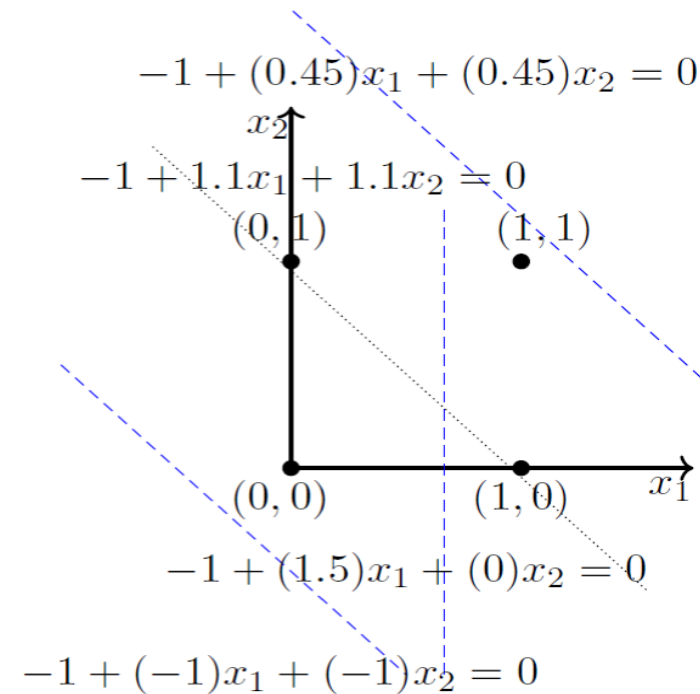


Error

- Let us fix the threshold ($-w_0 = 1$) and try different values of w_1, w_2
- Say, $w_1 = -1, w_2 = -1$
- What is wrong with this line? We make an error on 1 out of the 4 inputs
- Lets try some more values of w_1, w_2 and note how many errors we make

w_1	w_2	errors
-1	-1	3
1.5	0	1
0.45	0.45	3

- We are interested in those values of w_0, w_1, w_2 which result in 0 error



Perceptron learning algorithm

- Consider two vectors $\mathbf{w}$ and $\mathbf{x}$

$$\mathbf{w} = [w_0, w_1, w_2, \dots, w_n]$$

$$\mathbf{x} = [1, x_1, x_2, \dots, x_n]$$

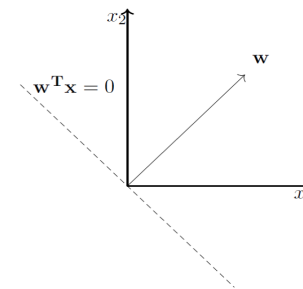
$$\mathbf{w} \cdot \mathbf{x} = \mathbf{w}^T \mathbf{x} = \sum_{i=0}^n w_i * x_i$$

- We can thus rewrite the perceptron rule as

$$y = 1 \quad \text{if} \quad \mathbf{w}^T \mathbf{x} \geq 0$$

$$= 0 \quad \text{if} \quad \mathbf{w}^T \mathbf{x} < 0$$

- We are interested in finding the line $\mathbf{w}^T \mathbf{x} = 0$ which divides the input space into two halves
- Every point ($\mathbf{x}$) on this line satisfies the equation $\mathbf{w}^T \mathbf{x} = 0$
- What can you tell about the angle (α) between $\mathbf{w}$ and any point ($\mathbf{x}$) which lies on this line?
- The angle is 90° ($\because \cos \alpha = \frac{\mathbf{w}^T \mathbf{x}}{\|\mathbf{w}\| \|\mathbf{x}\|} = 0$)
- Since the vector $\mathbf{w}$ is perpendicular to every point on the line it is actually perpendicular to the line itself



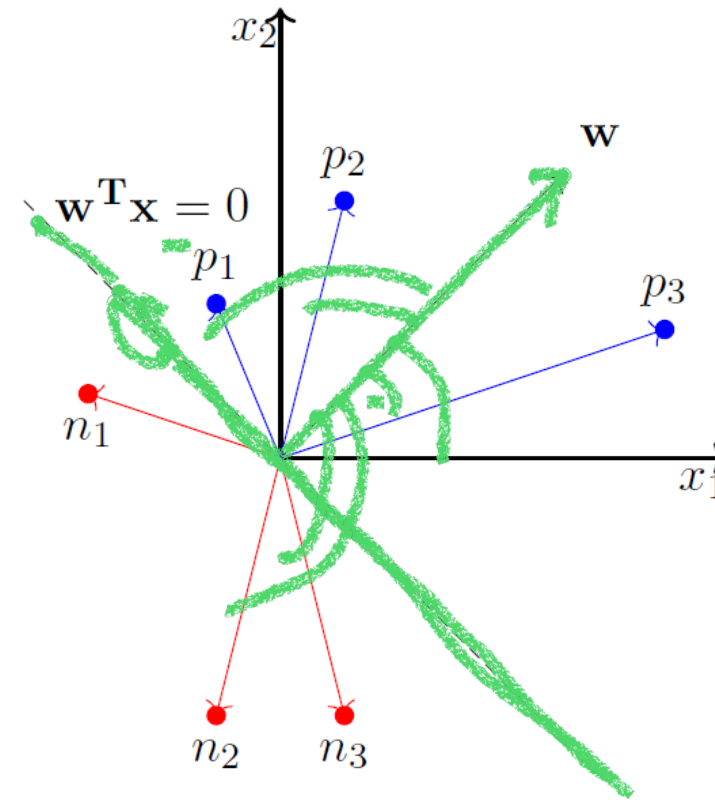
$A \cdot B = |A||B|\cos \theta$
dot product of two vectors

Consider some points (vectors) which lie in the positive half space of this line (*i.e.*, $\mathbf{w}^T \mathbf{x} \geq 0$)

What will be the angle between any such vector and $\mathbf{w}$? Obviously, less than 90°

What about points (vectors) which lie in the negative half space of this line (*i.e.*, $\mathbf{w}^T \mathbf{x} < 0$)

What will be the angle between any such vector and $\mathbf{w}$? Obviously, greater than 90°



Algorithm: Perceptron Learning Algorithm

$P \leftarrow \text{inputs with label } 1;$

$N \leftarrow \text{inputs with label } 0;$

Initialize $\mathbf{w}$ randomly;

while !convergence **do**

 Pick random $\mathbf{x} \in P \cup N$;

if $\mathbf{x} \in P$ and $\mathbf{w} \cdot \mathbf{x} < 0$ **then**

$\mathbf{w} = \mathbf{w} + \mathbf{x}$;

end

if $\mathbf{x} \in N$ and $\mathbf{w} \cdot \mathbf{x} \geq 0$ **then**

$\mathbf{w} = \mathbf{w} - \mathbf{x}$;

end

end

//the algorithm converges when all the
inputs are classified correctly

$$\cos \alpha = \frac{\mathbf{w}^T \mathbf{x}}{\|\mathbf{w}\| \|\mathbf{x}\|}$$

- For $\mathbf{x} \in P$ if $\mathbf{w} \cdot \mathbf{x} < 0$ then it means that the angle (α) between this $\mathbf{x}$ and the current $\mathbf{w}$ is greater than 90° (but we want α to be less than 90°)
- What happens to the new angle (α_{new}) when $\mathbf{w}_{\text{new}} = \mathbf{w} + \mathbf{x}$

$$\cos(\alpha_{\text{new}}) \propto \mathbf{w}_{\text{new}}^T \mathbf{x}$$

$$\propto (\mathbf{w} + \mathbf{x})^T \mathbf{x}$$

$$\propto \mathbf{w}^T \mathbf{x} + \mathbf{x}^T \mathbf{x}$$

$$\propto \cos \alpha + \mathbf{x}^T \mathbf{x}$$

$$\cos(\alpha_{\text{new}}) > \cos \alpha$$

- Thus α_{new} will be less than α and this is exactly what we want

P

$\mathbf{w} < 0$
 > 90

Algorithm: Perceptron Learning Algorithm

$P \leftarrow \text{inputs with label } 1;$

$N \leftarrow \text{inputs with label } 0;$

Initialize $\mathbf{w}$ randomly;

while !convergence **do**

 Pick random $\mathbf{x} \in P \cup N$;

if $\mathbf{x} \in P$ and $\mathbf{w} \cdot \mathbf{x} < 0$ **then**

$\mathbf{w} = \mathbf{w} + \mathbf{x}$;

end

if $\mathbf{x} \in N$ and $\mathbf{w} \cdot \mathbf{x} \geq 0$ **then**

$\mathbf{w} = \mathbf{w} - \mathbf{x}$;

end

end

//the algorithm converges when all the
inputs are classified correctly

$$\cos \alpha = \frac{\mathbf{w}^T \mathbf{x}}{\|\mathbf{w}\| \|\mathbf{x}\|}$$

- For $\mathbf{x} \in N$ if $\mathbf{w} \cdot \mathbf{x} \geq 0$ then it means that the angle (α) between this $\mathbf{x}$ and the current $\mathbf{w}$ is less than 90° (but we want α to be greater than 90°)
- What happens to the new angle (α_{new}) when $\mathbf{w}_{new} = \mathbf{w} - \mathbf{x}$

$$\begin{aligned}\cos(\alpha_{new}) &\propto \mathbf{w}_{new}^T \mathbf{x} \\ &\propto (\mathbf{w} - \mathbf{x})^T \mathbf{x} \\ &\propto \mathbf{w}^T \mathbf{x} - \mathbf{x}^T \mathbf{x} \\ &\propto \cos \alpha - \mathbf{x}^T \mathbf{x}\end{aligned}$$

$$\cos(\alpha_{new}) < \cos \alpha$$

- Thus α_{new} will be greater than α and this is exactly what we want

Working of perceptron algorithm

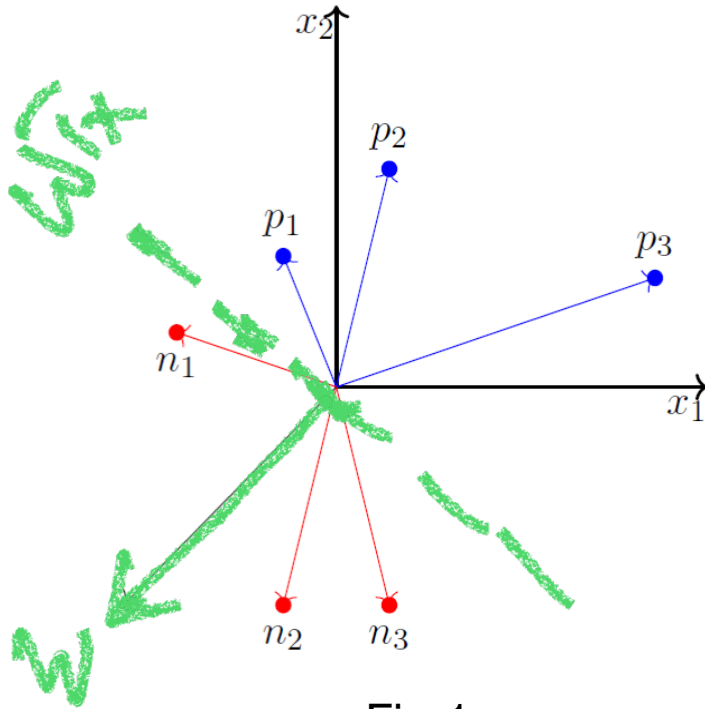


Fig 1

- We initialized $\mathbf{w}$ to a random value
- We observe that currently, $\mathbf{w} \cdot \mathbf{x} < 0$ ($\because$ angle $> 90^\circ$) for all the positive points and $\mathbf{w} \cdot \mathbf{x} \geq 0$ ($\because$ angle $< 90^\circ$) for all the negative points (the situation is exactly opposite of what we actually want it to be)
- We now run the algorithm by randomly going over the points
- Randomly pick a point (say, p_1), apply correction $\mathbf{w} = \mathbf{w} + \mathbf{x} \because \mathbf{w} \cdot \mathbf{x} < 0$ (you can check the angle visually)

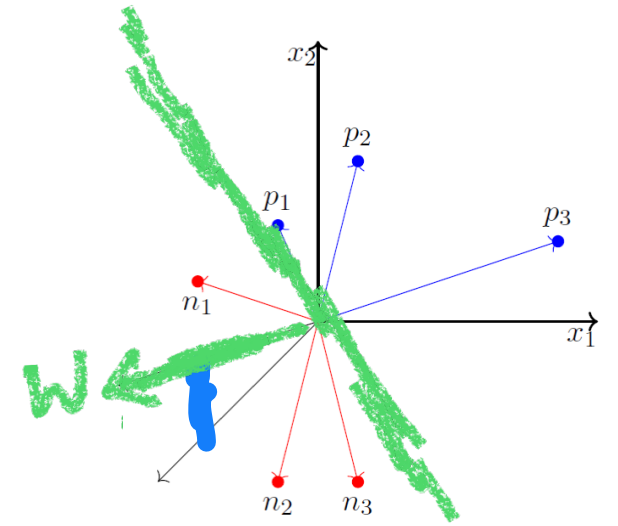
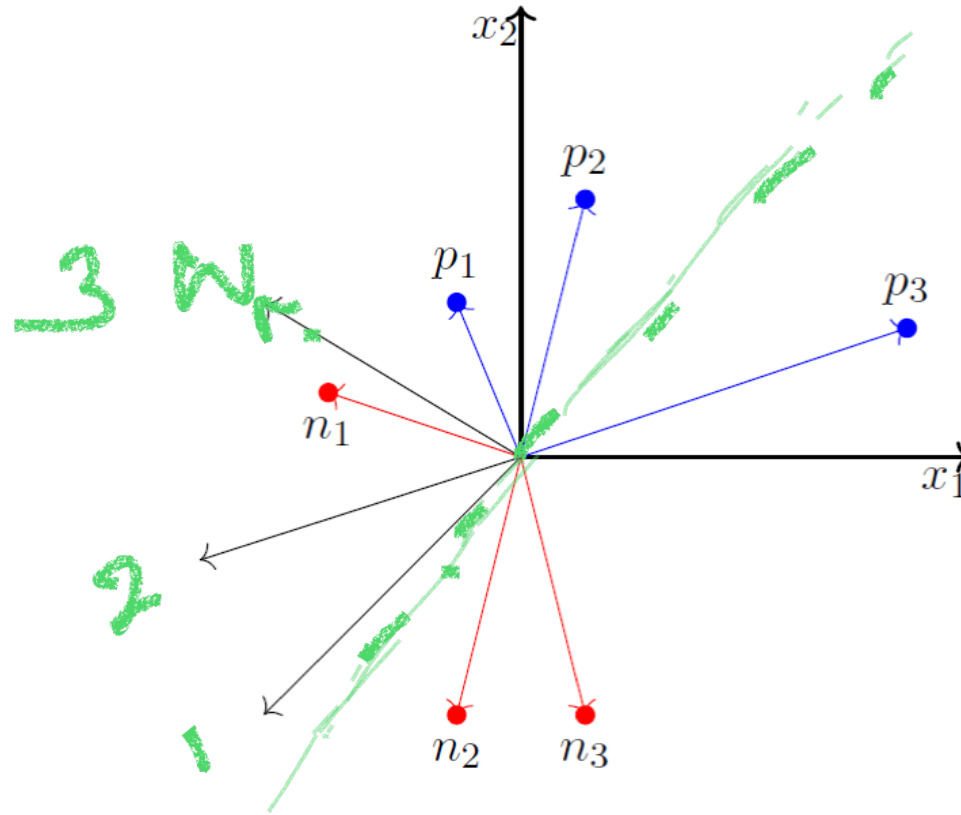
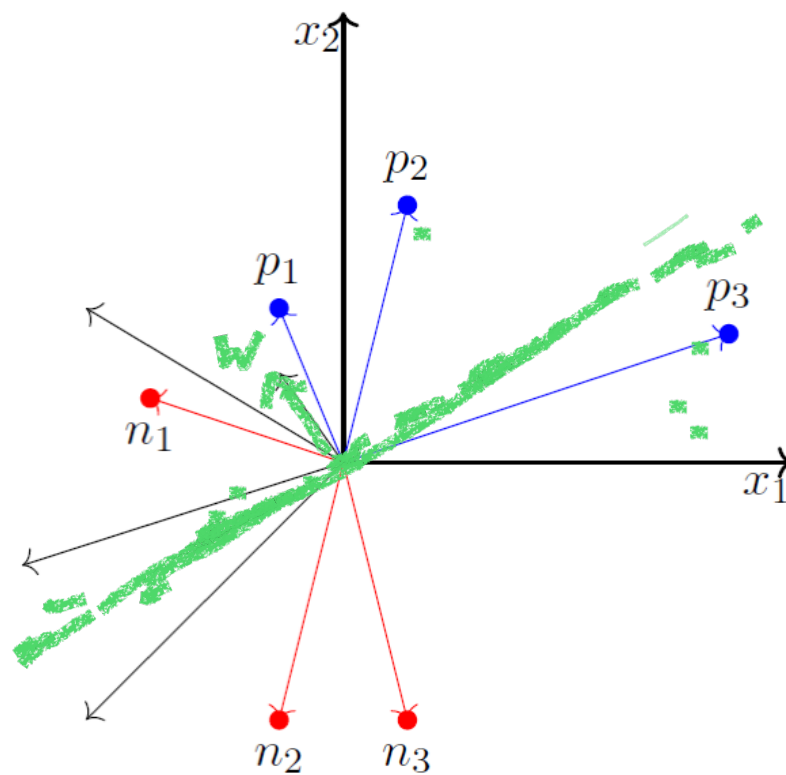


Fig 2

- Randomly pick a point (say, p_2), apply correction $\mathbf{w} = \mathbf{w} + \mathbf{x} \because \mathbf{w} \cdot \mathbf{x} < 0$ (you can check the angle visually)



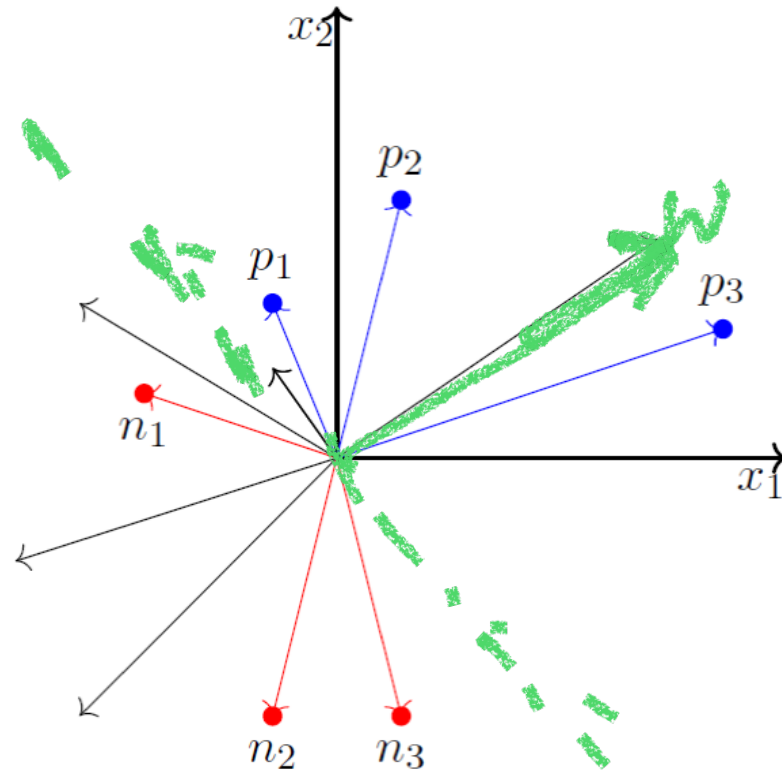
Randomly pick a point (say, n_1), apply correction $\mathbf{w} = \mathbf{w} - \mathbf{x} \because \mathbf{w} \cdot \mathbf{x} \geq 0$ (you can check the angle visually)



Randomly pick a point (say, n_3), no correction needed since $w \cdot x < 0$

Randomly pick a point (say, n_2), no correction needed since $w \cdot x < 0$

Randomly pick a point (say, p_3), apply correction $w = w + x$ since $w \cdot x < 0$



Randomly pick a point (say, p_1), no correction needed since $w \cdot x \geq 0$

The algorithm has converged

$w \cdot x$ is perpendicular to w

no change in one iteration over all points

Perceptron

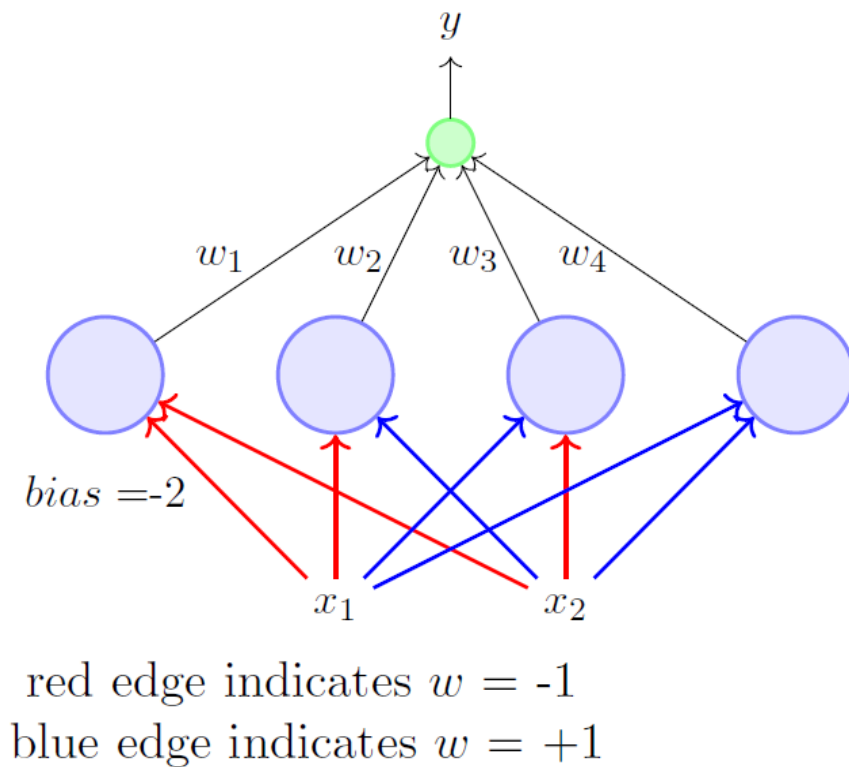
- Real valued inputs are allowed in perceptron
- we can learn the threshold
- A perceptron allows weights to be assigned to inputs to give more importance to some weights
- The functions which are not linearly separable are not possible with a single perceptron .
But it is possible with Network of Perceptrons

Implementation of any Boolean functions using a network of perceptrons

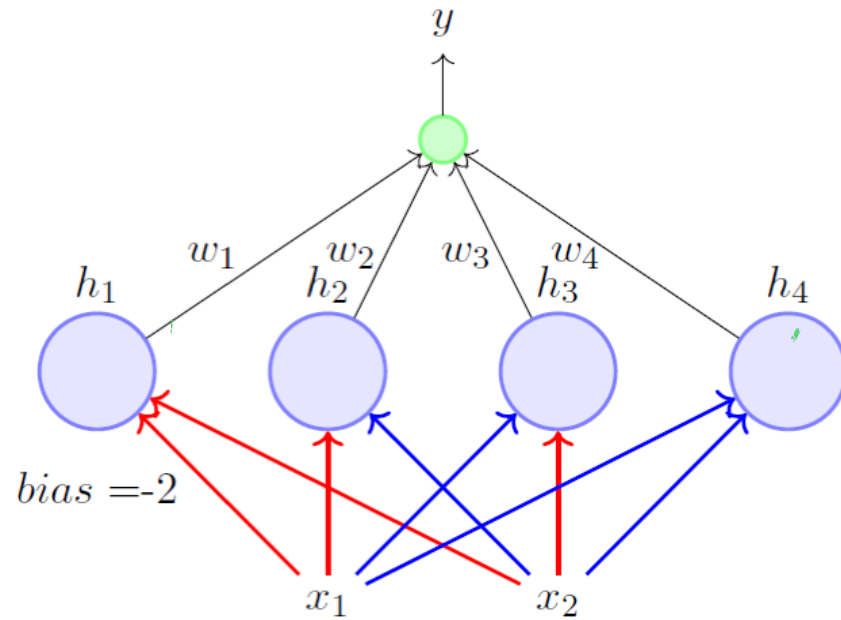
- How many boolean functions can you design from 2 inputs ?
- Let us begin with some easy ones which you already know ..

x_1	x_2	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}	f_{16}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

- Of these, how many are linearly separable ? (turns out all except XOR and !XOR - feel free to verify)
- In general, how many boolean functions can you have for n inputs ? 2^{2^n}
- How many of these 2^{2^n} functions are not linearly separable ? For the time being, it suffices to know that at least some of these may not be linearly inseparable (I encourage you to figure out the exact answer :-)



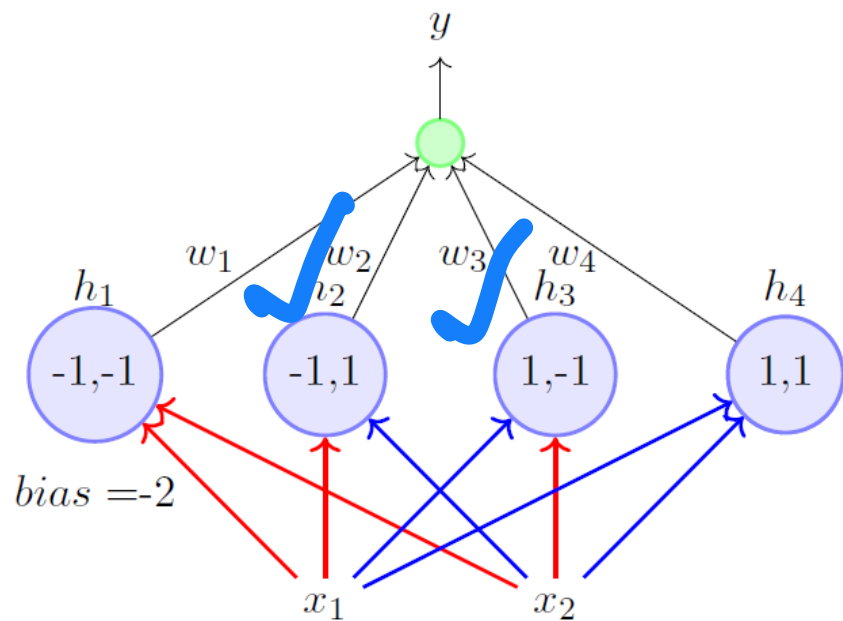
- For this discussion, we will assume True = +1 and False = -1
- We consider 2 inputs and 4 perceptrons
- Each input is connected to all the 4 perceptrons with specific weights
- The bias (w_0) of each perceptron is -2 (i.e., each perceptron will fire only if the weighted sum of its input is ≥ 2)
- Each of these perceptrons is connected to an output perceptron by weights (which need to be learned)
- The output of this perceptron (y) is the output of this network



red edge indicates $w = -1$
blue edge indicates $w = +1$

Terminology:

- This network contains 3 layers
- The layer containing the inputs (x_1, x_2) is called the **input layer**
- The middle layer containing the 4 perceptrons is called the **hidden layer**
- The final layer containing one output neuron is called the **output layer**
- The outputs of the 4 perceptrons in the hidden layer are denoted by h_1, h_2, h_3, h_4
- The red and blue edges are called layer 1 weights
- w_1, w_2, w_3, w_4 are called layer 2 weights



red edge indicates $w = -1$
blue edge indicates $w = +1$

We claim that this network can be used to implement **any** boolean function (linearly separable or not) !

In other words, we can find w_1, w_2, w_3, w_4 such that the truth table of any boolean function can be represented by this network

Each perceptron in the middle layer fires only for a specific input (and no two perceptrons fire for the same input)

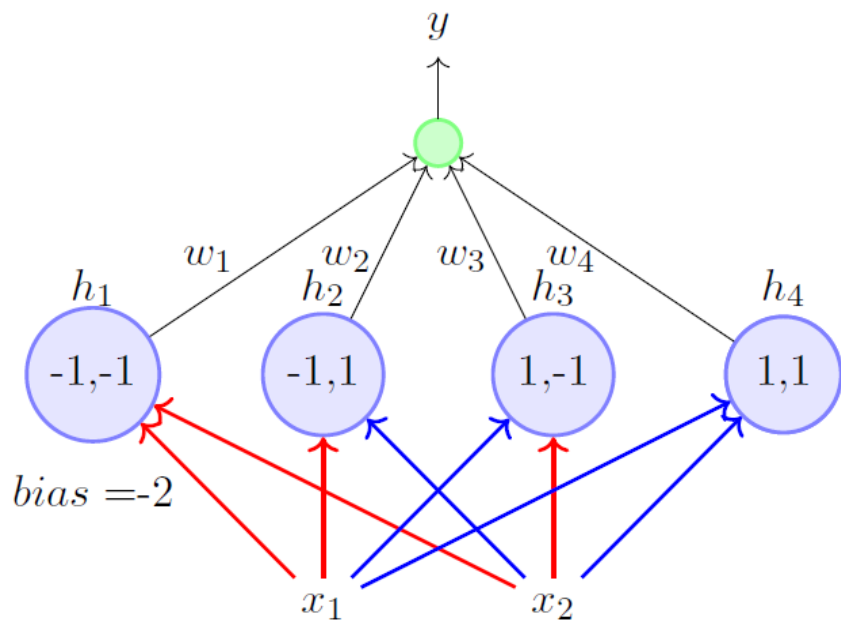
the first perceptron fires for $\{-1, -1\}$

the second perceptron fires for $\{-1, 1\}$

the third perceptron fires for $\{1, -1\}$

the fourth perceptron fires for $\{1, 1\}$

Let us see why this network works by taking an example of the XOR function



red edge indicates $w = -1$
blue edge indicates $w = +1$

- Let w_0 be the bias output of the neuron (i.e., it will fire if $\sum_{i=1}^4 w_i h_i \geq w_0$)

x_1	x_2	XOR	h_1	h_2	h_3	h_4	$\sum_{i=1}^4 w_i h_i$
0	0	0	1	0	0	0	w_1
0	1	1	0	1	0	0	w_2
1	0	1	0	0	1	0	w_3
1	1	0	0	0	0	1	w_4

- This results in the following four conditions to implement XOR: $w_1 < w_0, w_2 \geq w_0, w_3 \geq w_0, w_4 < w_0$
- Essentially each w_i is now responsible for one of the 4 possible inputs and can be adjusted to get the desired output for that input
- It should be clear that the same network can be used to represent the remaining 15 boolean functions also
- Each boolean function will result in a different set of non-contradicting inequalities which can be satisfied by appropriately setting w_1, w_2, w_3, w_4

Assignment 1

Using the same network represent the remaining 15

Boolean functions (Each boolean function will result in a different set of non-contradicting inequalities which can be satisfied by appropriately setting w_1, w_2, w_3, w_4)

Example (3 inputs)

Any boolean function of n inputs can be represented exactly by a network of perceptrons containing 1 hidden layer with 2^n perceptrons and one output layer containing 1 perceptron

